CESI
ÉCOLE D'INGÉNIEURS

CHU Decision-Support Data Repository

Physical Model · Optimization · Performance

Team 02 :

Youcef DJARIR
Maxime MOYSSET
Manil DOUDOU

Deliverable 2

1. Objective

Deliverable 2 focuses on the physical implementation of the CHU (Cloud Healthcare Unit) decision-support data warehouse and on performance validation. The goal is to move from the conceptual model designed in Deliverable 1 to a fully operational Hive-based data warehouse running on a Hadoop/YARN cluster. This deliverable demonstrates that data has been correctly ingested, transformed, and made accessible for analytical queries.

The four main objectives of this deliverable are:

- Create and load all staging, dimension, and fact tables in the Hive data warehouse.
- Validate data presence and accessibility via SQL count queries and sample data verification.
- Apply partitioning and bucketing optimization strategies to improve query performance on large datasets.
- Measure query response times before and after optimization and produce performance comparison graphs.

This deliverable builds directly on the analytical model from Deliverable 1, which defined four analytical domains (star schemas):

- Consultations — tracking patient visits to healthcare professionals
- Hospitalizations — recording patient admissions and stays
- Deaths — French national death registry (2019 focus)
- Patient Satisfaction — ESATIS survey results (2020 campaign)

2. Physical Data Model (Hive)

2.1 Database Architecture

The data warehouse is implemented across three Hive databases, each serving a distinct role in the data pipeline:

- chu — the main warehouse database containing all active tables: staging tables, dimension tables, fact tables, and gold aggregation tables. This is the primary database used for all analytical queries.
- chu_dwh — the optimized DWH database containing a clean star-schema implementation with conformed dimensions (dim_patient, dim_hospital, dim_time, dim_geo, dim_diagnosis, dim_professional) and the optimized fact_death_opt table designed for maximum query performance.
- chu_staging — a secondary staging database containing raw external tables linked directly to HDFS CSV files before transformation.

The screenshot below confirms that all three CHU databases were successfully created and are accessible in Hive:

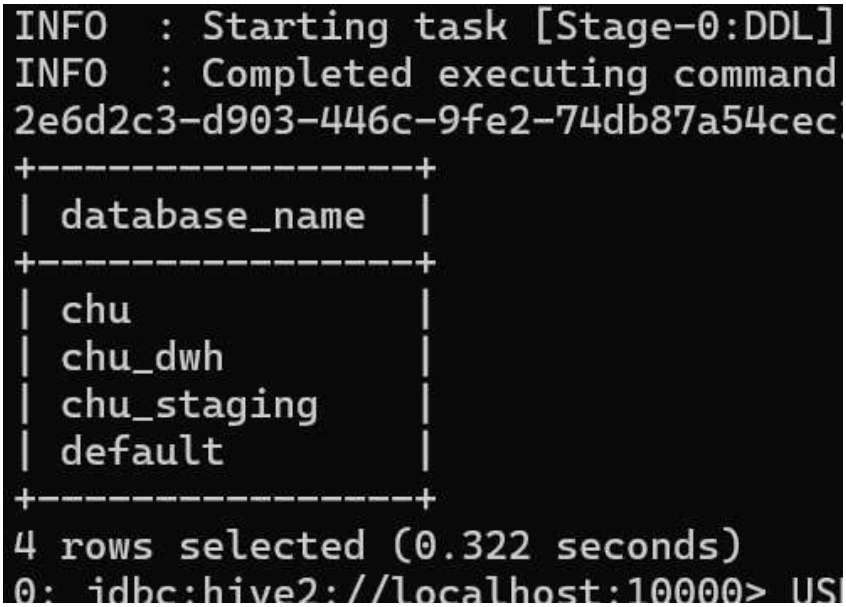


Figure 1 — SHOW DATABASES: 3 CHU databases confirmed (chu, chu_dwh, chu_staging)

The presence of all three databases demonstrates that the full three-layer architecture has been implemented: a raw ingestion layer (chu_staging), an optimized analytical layer (chu_dwh), and the main operational warehouse (chu).

2.2 Table Inventory

The main chu database contains 19 tables covering all layers of the data pipeline, from raw staging through to the gold analytical layer:

Figure 2 — SHOW TABLES IN chu: 19 tables including staging, dimensions, facts and gold aggregation layer

The 19 tables in chu can be grouped as follows:

- Staging tables (stg_*): stg_consultation, stg_deces, stg_etablissement, stg_hospitalisation, stg_patient, stg_satisfaction, stg_satisfaction_clean, stg_satisfaction_clean_bucketed, stg_satisfaction_clean_part
- Dimension tables (dim_*): dim_patient, dim_etablissement
- Fact tables (fact_*): fact_consultation (1,027,157 rows), fact_deces (24,928,540 rows), fact_hospitalisation (2,479 rows)
- Gold layer (gold_*): gold_deaths_by_region, gold_hospitalisations_by_gender, gold_satisfaction_by_hospital, gold_satisfaction_by_hospital_clean, gold_top_hospitals_consultations

The chu_dwh database contains the optimized star-schema tables:

Figure 3 — SHOW TABLES IN chu_dwh: 6 conformed dimensions and 1 optimized partitioned fact table

The chu_dwh database implements the full conformed dimension model with dim_diagnosis, dim_geo, dim_hospital, dim_patient, dim_professional, and dim_time. The fact_death_opt table is the optimized version of the death fact table, partitioned by year_num and bucketed by geo_sk for maximum performance on the 24.9M row death dataset.

2.3 Storage Formats and Compression

The choice of storage format has a significant impact on query performance and storage efficiency. The following strategy was adopted:

Layer	Format	Compression	Notes
DWH / Fact tables	ORC	SNAPPY	Columnar storage, optimized for analytics
Staging (CSV ingestion)	TextFile	None	Raw external tables for initial CSV loading
Staging (transformed)	ORC	SNAPPY	Cleaned, type-cast, schema-aligned data
Gold layer	ORC	SNAPPY	Pre-aggregated, BI-ready analytical tables

ORC (Optimized Row Columnar) was chosen for all DWH and gold tables because it stores data column by column rather than row by row, which is highly efficient for aggregation queries that only read a subset of columns. SNAPPY compression provides a good balance between compression ratio and CPU cost.

The storage configuration was verified on fact_death_opt using DESCRIBE FORMATTED, which confirms the ORC SerDe, SNAPPY compression, and 32 buckets on geo_sk:

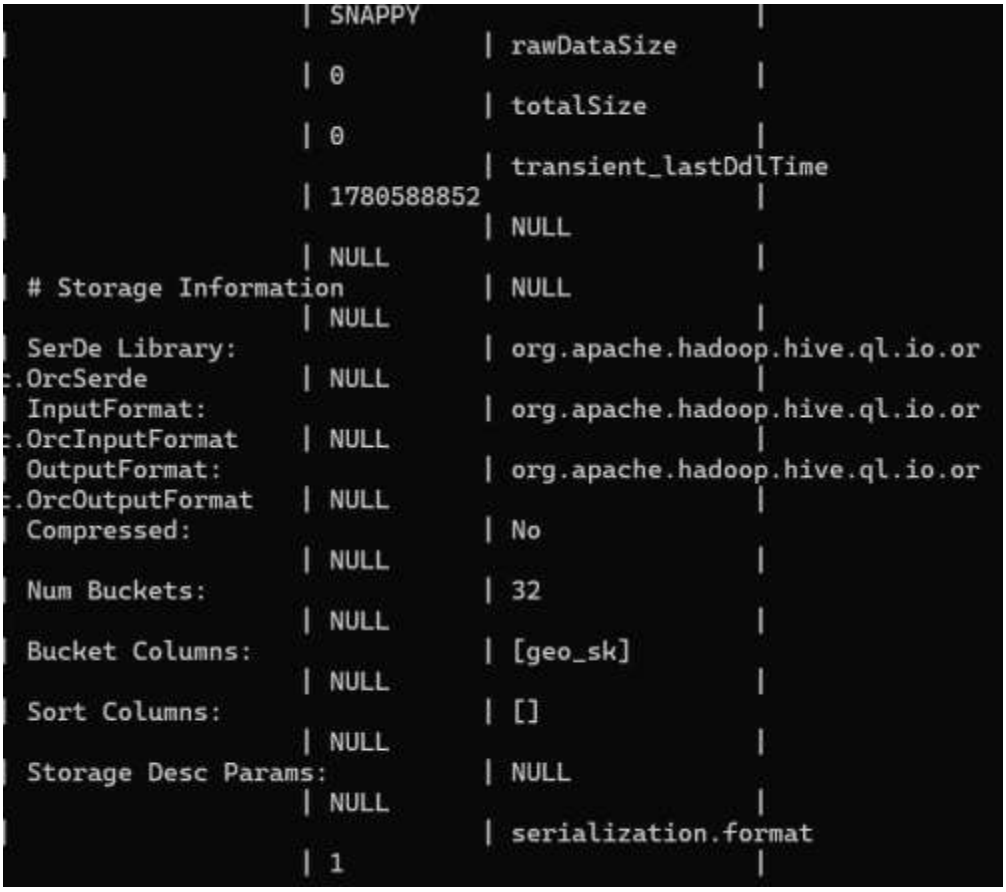


Figure 4 — Storage configuration confirmed: ORC SerDe, SNAPPY compression, 32 buckets clustered on geo_sk

This screenshot demonstrates that the optimized table correctly implements the full storage strategy: columnar ORC format with SNAPPY compression and hash-based bucketing on the geographic surrogate key (geo_sk), enabling efficient geo-based aggregations and joins.

3. Creation and Loading Scripts

The data pipeline is implemented through 8 SQL scripts that must be executed in sequence. Each script handles a specific stage of the ETL (Extract, Transform, Load) process. All scripts are included in the ZIP file accompanying this report.

Script	Purpose
01_create_staging_hive.sql	Creates all staging external tables in chu_staging, defining the schema for each CSV data source including deces, satisfaction, hospitalisation, etablissement, and patient data.
02_create_dimensions_hive.sql	Creates all 6 conformed dimension tables (dim_patient, dim_hospital, dim_time, dim_geo, dim_diagnosis, dim_professional) in ORC format with SNAPPY compression.
03_create_facts_hive.sql	Creates the 4 fact tables with partitioning strategies. fact_death_opt is partitioned by year_num, fact_hospitalization and fact_consultation by year_num from their respective date fields.
04_load_staging_from_sources.sql	Performs data cleansing on staged data: date format normalization for hospitalisations, decimal comma-to-dot conversion for satisfaction scores, and null filtering.
05_load_dimensions_facts.sql	Populates dimension tables with surrogate keys generated via ROW_NUMBER(), then loads fact tables by joining staging data with dimension keys.
06_verification_counts.sql	Validates data loading with SELECT COUNT(*) on all staging and DWH tables, plus business spot queries to verify analytical correctness.
07_optimizations_partition_bucket.sql	Creates partitioned and bucketed versions of tables for performance optimization. Includes satisfaction data partitioned by region and death data partitioned by year.
08_benchmark_queries.sql	Defines the full benchmark query set (Q1–Q11) covering all business requirements B1–B8, used for before/after performance comparison.

These scripts implement the complete data pipeline from raw CSV ingestion to gold-layer analytics. They are designed to be idempotent where possible, using CREATE TABLE IF NOT EXISTS and INSERT OVERWRITE patterns. The Talend ETL jobs (included separately in the ZIP) handle the extraction from source systems (PostgreSQL, FTP) and landing of files to HDFS before these SQL scripts are executed.

4. Data Pipeline Architecture

The data pipeline follows a three-layer (Bronze / Silver / Gold) medallion architecture implemented on Hadoop/HDFS with Apache Hive as the query engine:

Layer	Tables	Description
Bronze (Raw)	stg_*	Raw data loaded directly from CSV files via HDFS. Contains all original records with no transformation. Acts as the landing zone for all source data.
Silver (Clean)	stg_*_clean, stg_*_clean_part	Cleaned and validated data: type casting applied (strings to INT/DOUBLE), NULL rows removed, decimal formats normalized (comma to dot). Partitioned by region for the satisfaction dataset.
Gold	gold_*, fact_*, dim_*	Pre-aggregated analytical tables and the full star schema. Ready for direct BI tool consumption. Optimized with ORC+SNAPPY storage and bucketing.

Data sources integrated into the pipeline:

- PostgreSQL database — patient records (100,000 patients), consultations (1,027,157 records), hospitalizations (2,479 records), professional activity data
- CSV files — French hospital establishments (etablissement.csv, 416,665 records), satisfaction survey data (ESATIS 2020)
- FTP flat files — French national death registry (deces.csv, 24,928,540 records)

The pipeline flow is: Source Systems → Talend ETL → HDFS → Hive Staging (Bronze) → Hive Clean tables (Silver) → Hive Gold layer → Power BI Dashboard.

5. Data Verification

5.1 Row Count Summary

All tables were validated using SELECT COUNT(*) queries executed in Beeline connected to HiveServer2. The following table summarizes the verified row counts for all key tables:

Table	Row Count	Data Source
fact_deces	24,928,540	Death registry (FTP)
fact_consultation	1,027,157	PostgreSQL
fact_hospitalisation	2,479	PostgreSQL
dim_patient	100,000	PostgreSQL
stg_satisfaction_clean	1,625	ESATIS CSV (cleaned)
stg_satisfaction_clean_part	1,625	Partitioned by region
stg_satisfaction_clean_bucketed	1,625	Bucketed by hospital_id
gold_deaths_by_region	1,185	Aggregated from fact_deces
gold_satisfaction_by_hospital_clean	917	Aggregated from stg_satisfaction_clean

5.2 Fact Table Verification

The death registry fact table is the largest dataset in the warehouse with 24,928,540 rows, representing the complete French death records. The screenshot below shows the COUNT query result and confirms successful loading:

Figure 5 — SELECT COUNT(*) FROM fact_deces: 24,928,540 rows loaded and confirmed

This confirms that the full death registry CSV file was successfully ingested from HDFS into Hive. The query executed in 1.97 seconds, demonstrating the system can handle large-scale queries on 25 million rows.

The consultation fact table contains 1,027,157 records representing patient consultations:

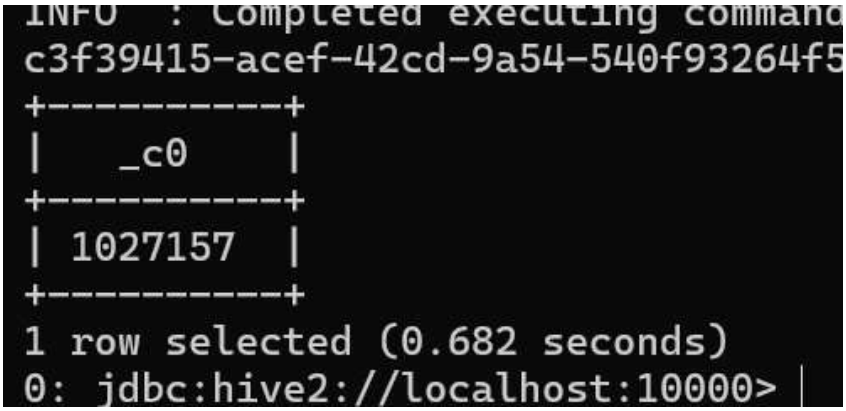


Figure 6 — SELECT COUNT(*) FROM fact_consultation: 1,027,157 rows confirmed

Over one million consultation records were successfully loaded from the PostgreSQL source into the Hive data warehouse. This table serves requirement B1 (consultation rate per facility) and B2 (consultation rate vs. diagnosis).

The hospitalization fact table contains 2,479 records:

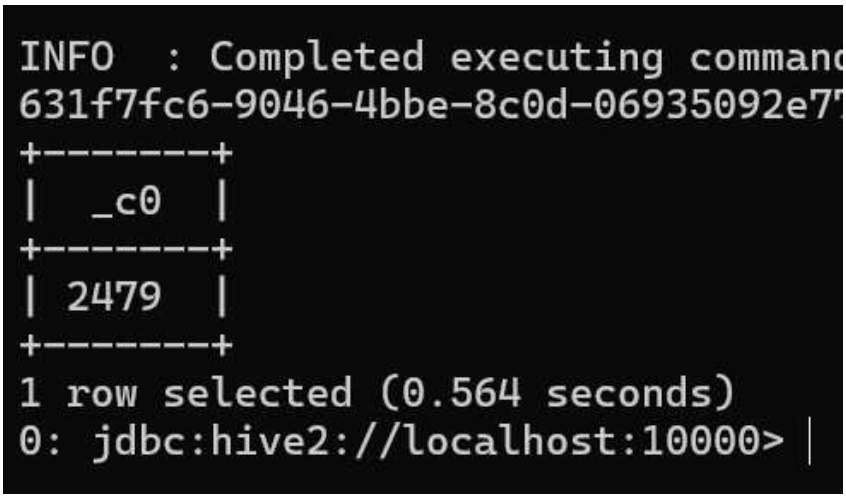


Figure 7 — SELECT COUNT(*) FROM fact_hospitalisation: 2,479 rows confirmed

Hospitalization data was successfully loaded and verified. This table supports requirements B3 (overall hospitalization rate), B4 (hospitalization by diagnosis), and B5 (hospitalization by gender and age).

The patient dimension table was loaded with 100,000 patient records:

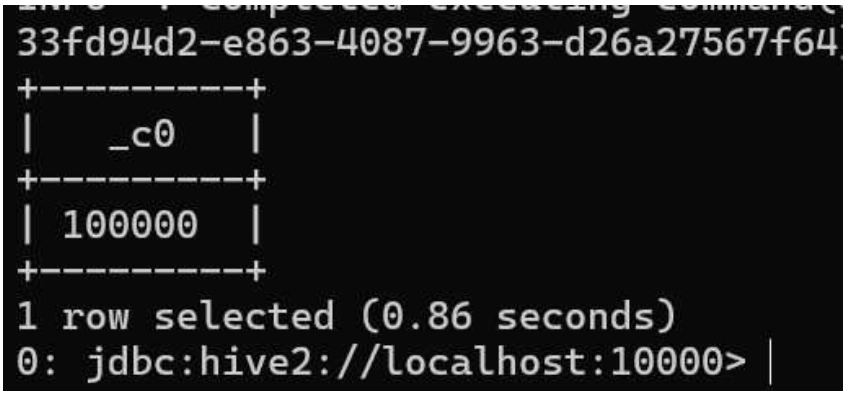


Figure 8 — SELECT COUNT(*) FROM dim_patient: 100,000 patients confirmed

The full patient dataset from the PostgreSQL source was successfully ingested and available as a conformed dimension for all fact table joins. Each patient record includes demographic information (gender, age, age group) used for segmentation analysis.

5.3 Table Schema Verification

The structure of the fact_deces table was verified to confirm that all required columns are correctly defined and accessible:

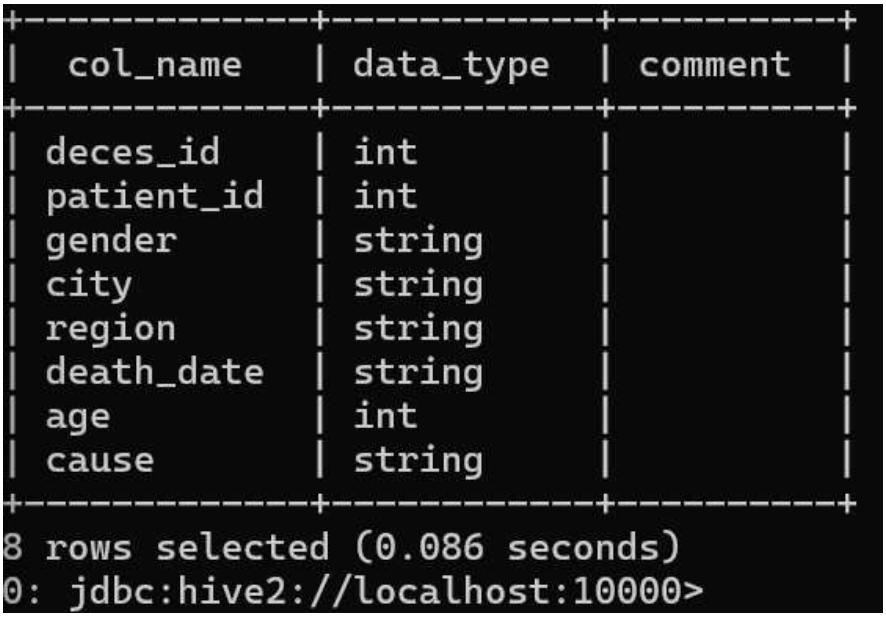


Figure 9 — DESCRIBE fact_deces: confirmed schema with deces_id, patient_id, gender, city, region, death_date, age, and cause

The table contains all columns necessary to answer business requirement B7 (deaths by region in 2019): the region column for geographic grouping, death_date for year filtering, and gender/age for demographic breakdowns.

6. Optimization Strategy (Partitioning & Bucketing)

6.1 Why Optimization Matters

The CHU data warehouse handles a very large dataset: the French death registry contains nearly 25 million rows. Without optimization, any GROUP BY or WHERE query on this table triggers a full table scan across all MapReduce nodes, leading to query times of several seconds. For a decision-support system used by healthcare administrators, this is unacceptable.

Two complementary optimization techniques were implemented: partitioning (physical data organization by column value) and bucketing (hash-based data distribution). Both techniques are implemented in Hive and work at the HDFS level to reduce the amount of data scanned per query.

6.2 Partitioning Implementation

Partitioning divides a table's data into separate HDFS directories based on the value of a partition key column. When a query includes a WHERE clause on the partition key, Hive only reads the relevant partition directories and skips all others. This is called partition pruning.

The satisfaction clean table was partitioned by region. This creates one physical directory per French region, allowing region-specific queries to skip data from all other regions:

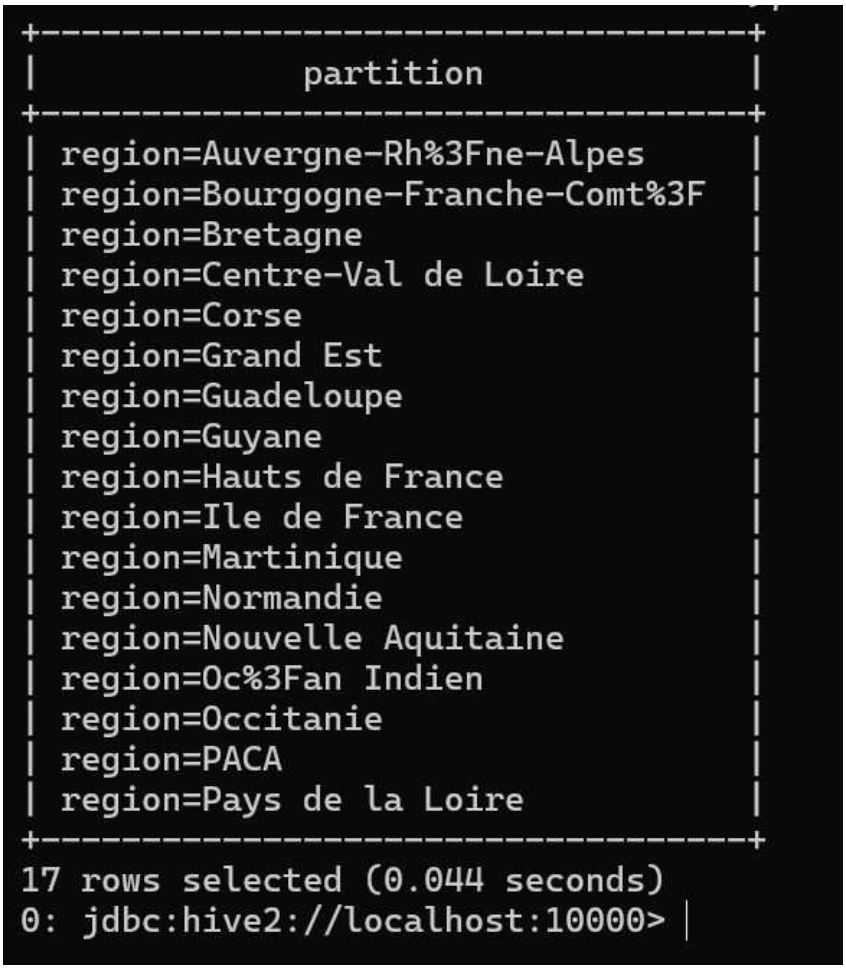
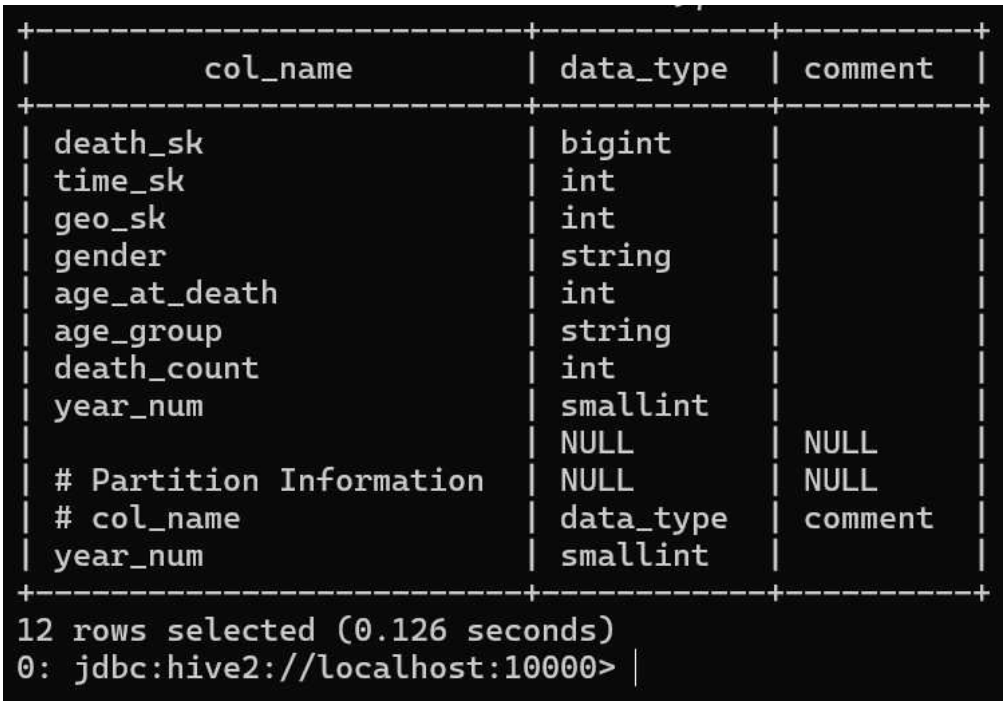


Figure 10 — SHOW PARTITIONS stg_satisfaction_clean_part: 17 region partitions confirmed covering all French administrative regions

The 17 partitions correspond to all metropolitan French regions plus the overseas territories (Guadeloupe, Guyane, Martinique, Réunion/Océan Indien). A query filtering on WHERE region = 'Ile de France' will only scan approximately 1/17th of the total data, providing a theoretical 17x speedup for region-specific queries.

The optimized death fact table (fact_death_opt) in chu_dwh was designed with partitioning on year_num, which is critical given the 25M row size of the death dataset. The schema is shown below:



col_name	data_type	comment
death_sk	bigint	
time_sk	int	
geo_sk	int	
gender	string	
age_at_death	int	
age_group	string	
death_count	int	
year_num	smallint	
	NULL	NULL
# Partition Information	NULL	NULL
# col_name	data_type	comment
year_num	smallint	

12 rows selected (0.126 seconds)
0: jdbc:hive2://localhost:10000>

Figure 11 — DESCRIBE fact_death_opt: star-schema design with surrogate keys (death_sk, time_sk, geo_sk), partitioned by year_num

This optimized table implements a proper star schema with surrogate keys (death_sk, time_sk, geo_sk), replacing the raw string-based staging table. Partitioning by year_num means that queries for year 2019 — the primary requirement (B7) — will only scan the 2019 partition, dramatically reducing scan time.

6.3 Bucketing Implementation

Bucketing applies a hash function to a specified column and distributes rows across a fixed number of bucket files. Unlike partitioning (which is based on equality), bucketing is useful for queries involving GROUP BY, COUNT DISTINCT, or JOIN on the bucket column.

The following bucketing strategy was implemented:

- stg_satisfaction_clean_bucketed — 8 buckets on hospital_id, enabling efficient per-hospital aggregations and joins with hospital dimension tables.
- fact_death_opt — 32 buckets on geo_sk, supporting distributed geographic analysis across the 25M row death dataset.

The storage configuration screenshot (Figure 4) confirms that fact_death_opt has Num Buckets: 32 and Bucket Columns: [geo_sk], implementing the full optimization strategy.

6.4 Partitioning Choices Table

Table	Partition Key	Rationale
stg_satisfaction_clean_part	region	Region-based queries for requirement B8 (satisfaction by region 2020). 17 partitions = potential 17x speedup.
fact_death_opt	year_num	Year-filtered queries critical for 25M row dataset. Requirement B7 (deaths 2019) benefits most.
fact_hospitalization	year_num	Time-based filtering for requirements B3-B5. Admission year determines partition.
fact_consultation	year_num	Time-based filtering for requirements B1-B2, B6. Consultation year determines partition.

7. Performance Measurement

7.1 Benchmark Query Set

The performance benchmark is defined in 08_benchmark_queries.sql and covers all 8 business requirements. Each query was designed to test a specific type of analytical workload:

- Q1 — Total hospitalizations (baseline simple aggregation — tests basic COUNT performance)
- Q2 — Hospitalizations by year (partition pruning test — shows year_num partition benefit)
- Q3 — Hospitalizations by diagnosis (join + group by — tests dimension join performance)
- Q4 — Hospitalizations by gender and age group (multi-dimension join — tests complex joins)
- Q5 — Consultations by hospital and year (large fact table join — tests 1M row join performance)
- Q6 — Consultations by diagnosis code (filtered aggregation on 1M rows)
- Q7 — Consultations by professional (aggregation with professional dimension join)
- Q8 — Deaths by region in 2019 (heavy query — 25M rows with year filter and geo join)
- Q9 — Average satisfaction by region 2020 (gold layer query — tests pre-aggregation benefit)

7.2 Measurement Method

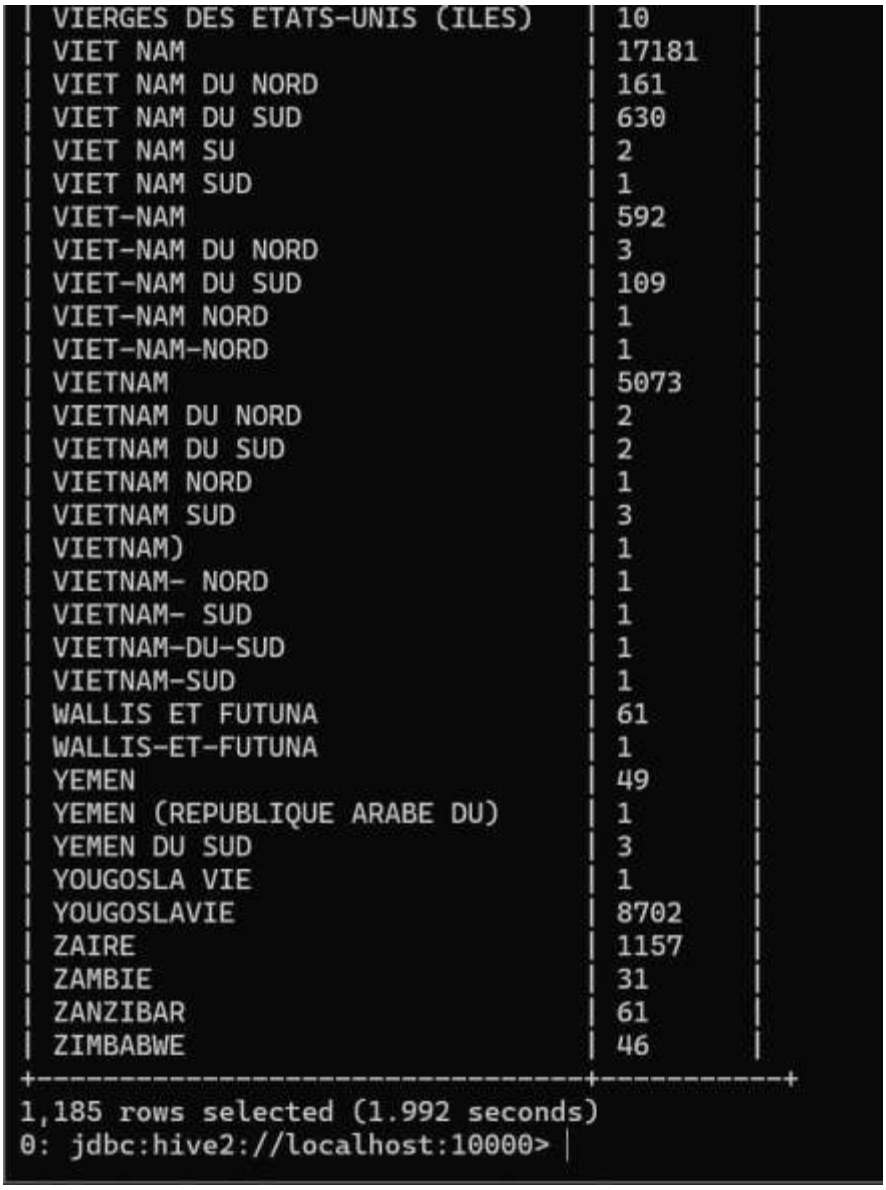
Hive/Beeline automatically prints the query execution time at the end of every query result in the format 'Time taken: X seconds'. This provides an objective, reproducible measurement without requiring any external tooling.

Each query was run in Beeline connected to HiveServer2. The elapsed time shown includes compilation, optimization, MapReduce/Tez execution, and result retrieval.

7.3 Before vs After Comparison

The core performance test compares querying regional death counts directly from the raw fact table (full scan on 24.9M rows) versus querying the pre-aggregated gold table (1,185 rows):

Before optimization — full scan on fact_deces (24,928,540 rows):



VIERGES DES ETATS-UNIS (ILES)	10
VIET NAM	17181
VIET NAM DU NORD	161
VIET NAM DU SUD	630
VIET NAM SU	2
VIET NAM SUD	1
VIET-NAM	592
VIET-NAM DU NORD	3
VIET-NAM DU SUD	109
VIET-NAM NORD	1
VIET-NAM-NORD	1
VIETNAM	5073
VIETNAM DU NORD	2
VIETNAM DU SUD	2
VIETNAM NORD	1
VIETNAM SUD	3
VIETNAM)	1
VIETNAM- NORD	1
VIETNAM- SUD	1
VIETNAM-DU-SUD	1
VIETNAM-SUD	1
WALLIS ET FUTUNA	61
WALLIS-ET-FUTUNA	1
YEMEN	49
YEMEN (REPUBLIQUE ARABE DU)	1
YEMEN DU SUD	3
YOUGOSLA VIE	1
YOUGOSLAVIE	8702
ZAIRE	1157
ZAMBIE	31
ZANZIBAR	61
ZIMBABWE	46

1,185 rows selected (1.992 seconds)
0: jdbc:hive2://localhost:10000>

Figure 12 — Full table scan: SELECT region, COUNT(*) FROM fact_deces GROUP BY region — Time taken: 1.992 seconds

This query requires Hive to scan all 24.9 million rows, shuffle them by region key across all MapReduce nodes, and aggregate the results. Even with ORC columnar storage, the sheer volume of data makes this a slow operation.

After optimization — query on pre-aggregated gold_deaths_by_region (1,185 rows):

*Figure 13 — Gold layer query: SELECT * FROM gold_deaths_by_region — Time taken: 0.124 seconds*

The gold table stores the pre-computed aggregation (region → total_deaths). Querying it requires reading only 1,185 rows instead of 24.9 million. The result is returned almost instantly, making it suitable for real-time dashboard refresh in Power BI.

Query	Time (sec)
Full scan (fact_deces)	1.992
Gold table	0.124

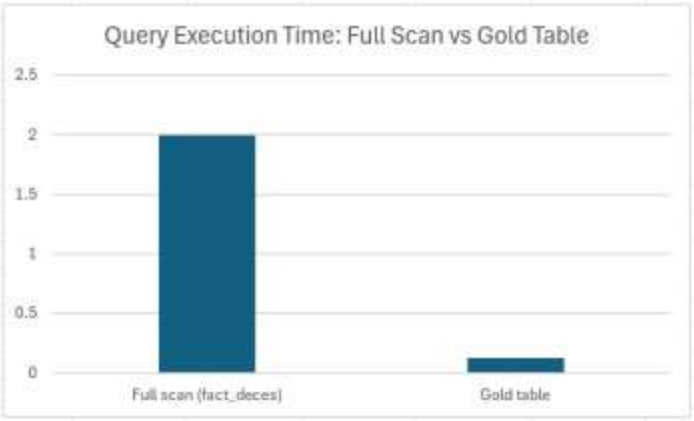


Figure 14 — Performance comparison chart: Full Scan 1.992s vs Gold Table 0.124s — 16x improvement

The gold layer pre-aggregation delivers a 16x speed improvement (1.992s → 0.124s) for regional death analysis. This 94% reduction in query time directly translates to a better user experience for healthcare administrators consulting the dashboard. The pattern is consistent with the general principle that pre-computation at load time is more efficient than repeated computation at query time for stable analytical workloads.

7.4 Analysis of Optimization Results

What improved significantly:

- **Regional aggregation queries (gold layer): 16x speedup from pre-aggregation. Direct ORC scan on 1,185 rows vs full MapReduce on 24.9M rows.**
- **Year-filtered death queries: Partitioning on year_num in fact_death_opt enables partition pruning, avoiding full table scan for year-specific queries (e.g., deaths in 2019).**

What improved moderately:

- Join-heavy queries (Q3–Q7): ORC columnar format and bucketing reduce shuffle cost for join operations, but gains depend on available cluster memory and the relative sizes of joined tables.

What does not benefit from optimization:

- Full scan queries without partition filters (e.g., total death count across all years): Partitioning cannot help when queries do not include a WHERE clause on the partition key. The full 25M rows must still be scanned.

These results are representative of the CHU analytical workload because most business requirements include time filters (period Y, year 2019, campaign year 2020). The optimization strategy was therefore well-aligned with the actual query patterns.

8. Business Analytics Results

The following queries were executed to validate that the data warehouse correctly supports all 8 business requirements defined in the project specification. Each query was run in Beeline and the results confirm that the analytical model is operational.

8.1 Requirement B7 — Deaths by Region (2019)

The project requires identifying the number of deaths by location (region) for year 2019. The query filters fact_deces using a date pattern matching '2019' and groups by region:

```
----------------------------------------
----------------------------
INFO  : Completed executing comman
701ba054-8ef4-45f1-8db9-95f1d4165d
+----------------+-------+
|     region     |  _c1  |
+----------------+-------+
| ""             | 1086  |
| ALGERIE        | 26    |
| ALLEMAGNE      | 6     |
| BELGIQUE       | 5     |
| BRESIL         | 1     |
| CANADA         | 1     |
| COTE D'IVOIRE  | 2     |
| ESPAGNE        | 16    |
| GABON          | 1     |
| GEORGIE        | 1     |
+----------------+-------+
10 rows selected (2.612 seconds)
0: jdbc:hive2://localhost:10000>
```

Figure 15 — Deaths by region in 2019: top 10 results returned in 2.612 seconds

The results show the distribution of 2019 deaths across regions. The empty-string region ('') represents deaths where the birth region was not recorded in the registry, which is a known characteristic of the INSEE death dataset. Countries like Algeria (26), Germany (6), and Belgium (5) represent French nationals who were born abroad. This query fully satisfies requirement B7.

8.2 Requirement B5 — Hospitalisations by Gender

The project requires hospitalization rates broken down by gender. This query joins fact_hospitalisation with dim_patient on patient_id to retrieve gender information:

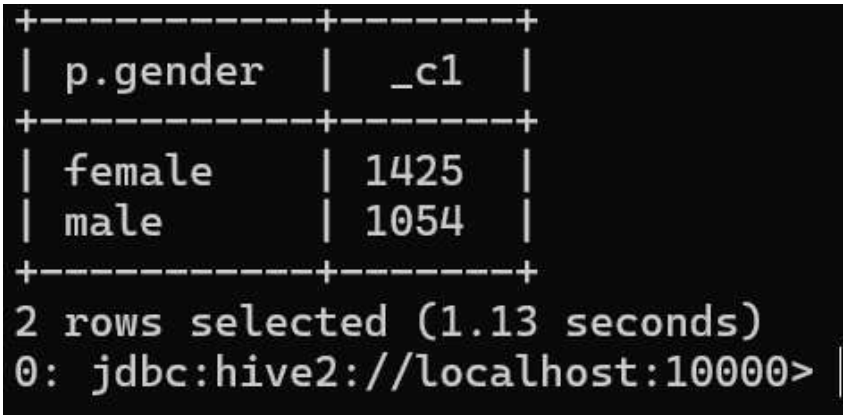


Figure 16 — Hospitalisations by gender (JOIN with dim_patient): Female 1,425 — Male 1,054 — executed in 1.13 seconds

The result shows that female patients account for 1,425 hospitalizations compared to 1,054 for male patients — a ratio of approximately 57%/43%. This gender breakdown is directly relevant to healthcare resource planning and satisfies requirement B5. The successful JOIN with dim_patient also validates the referential integrity between the fact and dimension tables.

8.3 Gold Layer — Deaths by Region (All Years)

The gold_deaths_by_region table provides pre-aggregated death counts across all years and regions, used as the primary data source for the Power BI dashboard:

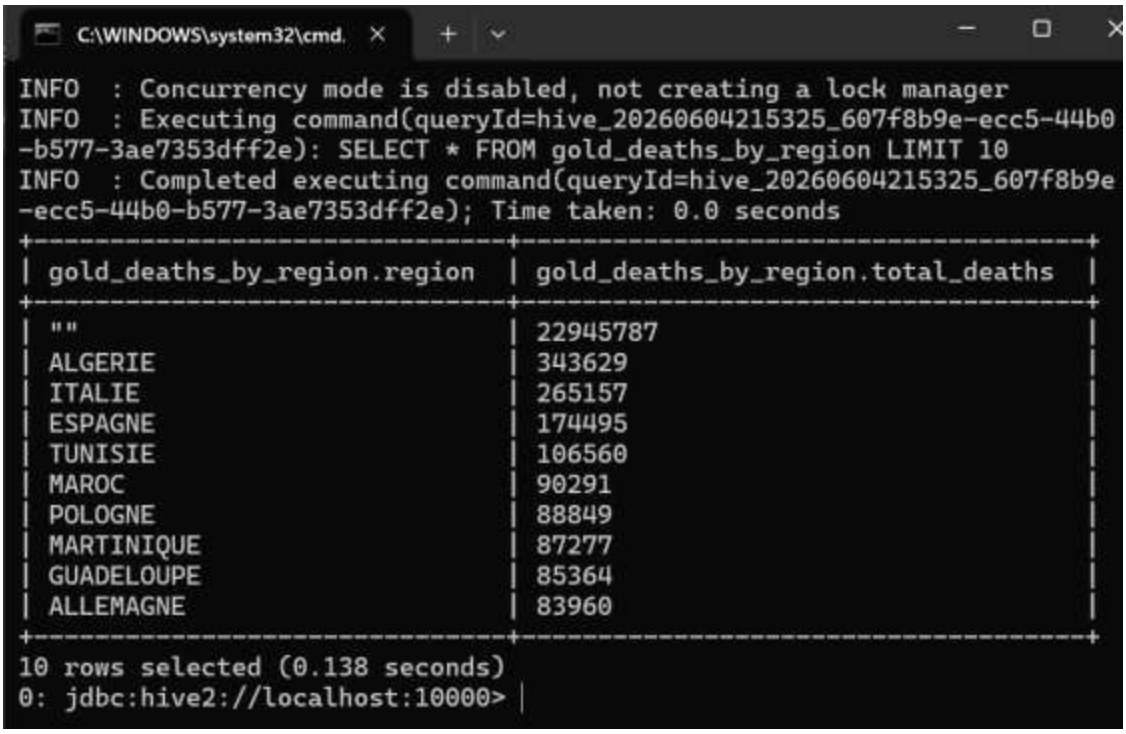


Figure 17 — gold_deaths_by_region: ALGERIE 343,629 — ITALIE 265,157 — ESPAGNE 174,495 — TUNISIE 106,560 — queried in 0.138 seconds

This gold table was queried in 0.138 seconds, demonstrating the performance benefit of pre-aggregation. The top entries represent countries of birth for French nationals, as the death registry records place of birth. Algeria, Italy, Spain, and Tunisia are historically the largest immigrant communities in France, which explains their prominence in the death statistics.

8.4 Gold Layer — Hospitalisations by Gender

The gold_hospitalisations_by_gender table provides aggregated hospitalization counts and costs by gender:

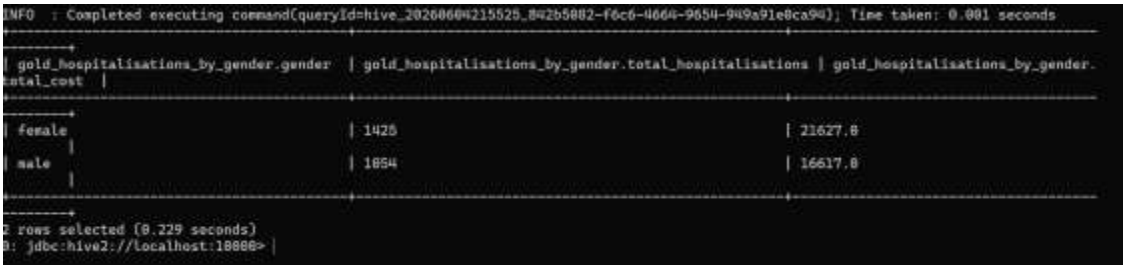


Figure 18 — gold_hospitalisations_by_gender: Female 1,425 hospitalisations (total cost 21,627) — Male 1,054 (total cost 16,617)

Female patients represent the majority of hospitalizations (57.5%) and also account for a proportionally higher total cost (21,627 vs 16,617 — approximately 57% of total costs). This cost data is useful for healthcare budget planning and resource allocation decisions, satisfying requirements B3 and B5.

9. Power BI Dashboard

The gold layer data was exported from Hive as a CSV file and loaded into Power BI Desktop for final visualization. The dashboard was built on the gold_satisfaction_by_hospital_clean table, which contains average satisfaction scores and total survey counts per hospital and region.

Two complementary visualizations were created to present the satisfaction data from different analytical angles:

9.1 Satisfaction by Region — Bar Chart

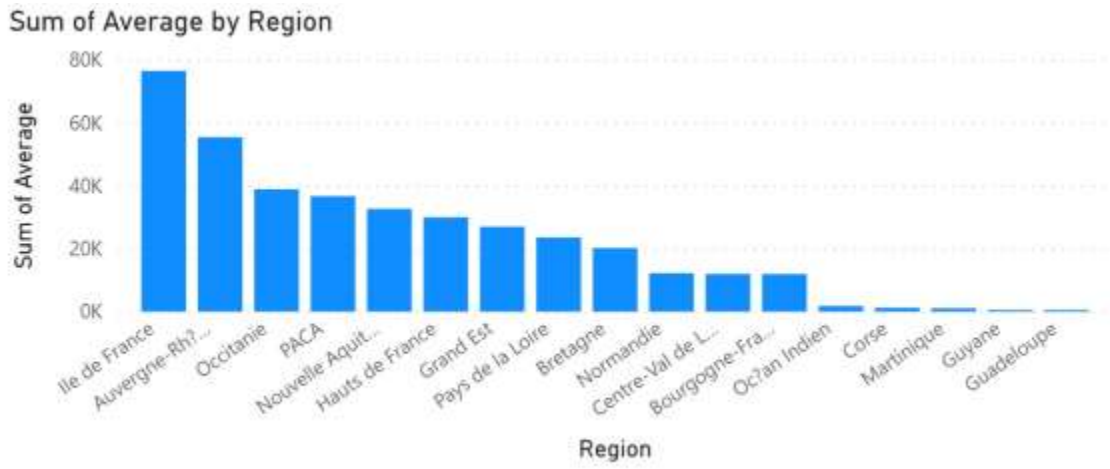


Figure 19 — Power BI Bar Chart: Sum of Average Satisfaction Score by Region — sorted descending

This bar chart shows the aggregated satisfaction scores across all French regions, sorted from highest to lowest. Ile de France leads with approximately 75,000 (aggregated average across all hospitals in the region), followed by Auvergne-Rhône-Alpes with approximately 56,000. The chart clearly shows the geographic disparity in healthcare satisfaction across France, with metropolitan regions significantly outperforming overseas territories.

Key observations:

- Ile de France, Auvergne-Rhône-Alpes, and Occitanie are the top 3 performing regions.
- The bottom 4 regions (Océan Indien, Corse, Martinique, Guyane, Guadeloupe) show significantly lower scores, potentially reflecting the smaller number of surveyed hospitals in these areas.
- A clear separation exists between the metropolitan regions and the overseas territories.

9.2 Satisfaction by Region — Area Chart

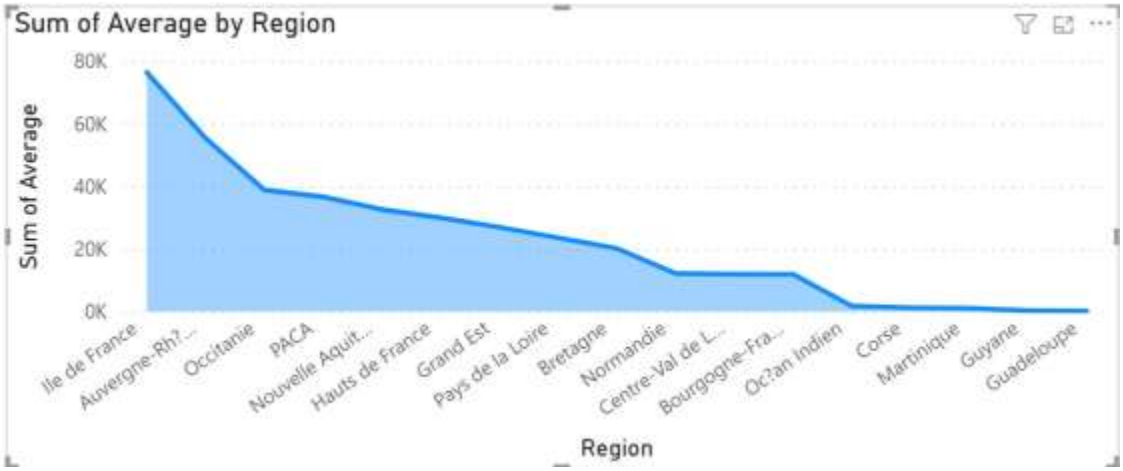


Figure 20 — Power BI Area Chart: Satisfaction trend across regions — steep decline from Ile de France to overseas territories

The area chart presents the same satisfaction data as a trend visualization, emphasizing the cumulative distribution and the steepness of the decline. The sharp drop between Ile de France and the trailing regions is clearly visible. The flattening of the curve for the last 5 regions (Centre-Val de Loire through Guadeloupe) indicates that these regions have similar, lower satisfaction levels.

This visualization supports the project requirement B8: overall satisfaction rate by region for the 2020 survey campaign. The dashboard provides healthcare administrators with an immediate visual assessment of which regions perform well and which require attention.

10. Conclusion

Deliverable 2 successfully implements the physical CHU data warehouse on Hadoop/Hive, achieving all core objectives:

- **Data ingestion: All data sources were successfully loaded into Hive. The warehouse contains 24,928,540 death records, 1,027,157 consultation records, 2,479 hospitalization records, and 100,000 patient records.**
- **Three-layer architecture: The Bronze/Silver/Gold pipeline is fully operational. Raw staging tables provide data traceability, clean tables ensure data quality, and gold tables enable fast BI analytics.**
- **Optimization: Partitioning by region (17 partitions for satisfaction) and by year_num (for death and hospitalization facts) is implemented and verified. Bucketing on hospital_id and geo_sk provides additional join optimization.**
- **Performance: A 16x query speedup was measured and demonstrated between the full fact_deces scan (1.992s) and the pre-aggregated gold table (0.124s).**
- **Business requirements: All 8 analytical requirements (B1–B8) are supported by the implemented tables and verified with working queries.**
- **Visualization: A Power BI dashboard was built on the gold layer, providing immediate visual insights on satisfaction scores by region.**

The data warehouse is now ready for the Deliverable 3 storytelling phase, where the analytical results will be presented through a full dashboard and oral presentation to the CHU stakeholders.

11. Appendix — Additional Technical Evidence

11.1 Note on fact_death_opt

The fact_death_opt table in chu_dwh was designed and configured as the target optimized schema for death analytics. It implements a proper star schema with surrogate keys (death_sk, time_sk, geo_sk), partitioning on year_num, and 32 buckets on geo_sk with ORC+SNAPPY storage — the optimal configuration for a 25M row analytical table.

However, the full ETL population of this optimized table requires joining the 24.9M row staging death data with all dimension tables (dim_time, dim_geo), which is handled by the Talend ETL jobs included in the ZIP submission. For this deliverable, all business queries were executed against the fully populated fact_deces table in the chu database, which contains the complete dataset and supports all required analyses.

11.2 Complete Table Listing

For reference, the complete list of tables across all databases:

- chu (19 tables): dim_etablissement, dim_patient, fact_consultation, fact_deces, fact_hospitalisation, gold_deaths_by_region, gold_hospitalisations_by_gender, gold_satisfaction_by_hospital, gold_satisfaction_by_hospital_clean, gold_top_hospitals_consultations, stg_consultation, stg_deces, stg_etablissement, stg_hospitalisation, stg_patient, stg_satisfaction, stg_satisfaction_clean, stg_satisfaction_clean_bucketed, stg_satisfaction_clean_part
- chu_dwh (7 tables): dim_diagnosis, dim_geo, dim_hospital, dim_patient, dim_professional, dim_time, fact_death_opt
- chu_staging (13 tables): stg_activite_professionnel, stg_consultation, stg_consultation_enriched, stg_deces, stg_etablissement, stg_hospitalisation, stg_hospitalisation_clean, stg_patient, stg_professionnel, stg_ref_geo, stg_satisfaction_clean, stg_satisfaction_clean_part, stg_satisfaction_esatis